

X-Engine: 针对大规模电商交易处理的优化存储引擎

黄贵、程训涛、王建英、王玉杰、何登城、张铁鹰、李飞飞、王胜、曹伟和李强 {qushan,xuntao.cxt,beilou.wjy,zhenneng.wyj,dengcheng.hedc, tieying.zhang,lifeifei,sh.wang,mingsong.cw,junyu}@alibaba-inc.com Alibaba Group

抽象的

阿里巴巴运营着全球最大的电子商务平台，为超过 6 亿客户提供服务，2018 财年 GMV（商品总价值）超过 7,680 亿美元。在线电子商务交易具有三个显著特征：

（1）随着重大销售和促销活动的开始，每秒交易量急剧增加；（2）大量的热门记录很容易淹没系统缓冲区；（3）由于在不同的时间段内不同类别的促销活动，不同记录的“温度”（热与冷）快速变化。例如，阿里巴巴的 OLTP 数据库集群在 2018 年双十一全球购物节开始时，交易量增长了 122 倍，每秒处理高达 491,000 笔销售交易，相当于每秒超过 7000 万笔数据库交易。为了应对这些挑战，我们引入了 X-Engine，这是阿里巴巴构建的 POLARDB 的写入优化存储引擎，它采用带有 LSM-tree（日志结构合并树）的分层存储架构，以利用 FPGA 加速压缩等硬件加速，以及一系列优化，包括事务中的异步写入、多级管道和压缩过程中的增量缓存替换。评估结果表明，在此类事务性工作负载下，X-Engine 的性能优于其他存储引擎。

CCS 概念

· 信息系统→数据访问方法； DBMS 引擎架构； 数据库事务处理。关键词

OLTP数据库、存储引擎、电子商务交易

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGMOD '19, June 30-July 5, 2019, Amsterdam, Netherlands

© 2019 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-5643-5/19/06.

<https://doi.org/10.1145/3299869.3314041>

ACM参考格式：

黄贵、程训涛、王建英、王玉杰、何登城、张铁鹰、李菲菲、王盛、曹伟和李强。2019. X-Engine: 面向大规模电子商务交易处理的优化存储引擎。2019年数据管理国际会议（SIGMOD'19），2019年6月30日至7月5日，荷兰阿姆斯特丹。ACM，美国纽约州纽约市，15 页。 <https://doi.org/10.1145/3299869.3314041>

1 简介

阿里巴巴运营着全球最大、最繁忙的电子商务平台，包括消费者对消费者零售市场淘宝、企业对企业市场天猫和其他在线市场，为超过6亿活跃消费者提供服务，2018财年GMV超过7680亿美元。这样的在线购物市场创造了新的购物和销售方式。例如，在线促销活动一开始就可以迅速扩大对全球客户的吸引力，因为与线下实体店相比，数字市场中的在线商店不受物理限制。

电子商务交易的处理是在线购物市场的支柱。我们发现此类交易具有三个主要特征：（1）随着主要销售和促销活动的开始，每秒交易量急剧增加；（2）大量的热记录很容易淹没系统缓冲区；（3）由于短时间内不同类别的促销活动的可用性，不同记录的“温度”（热与冷）快速变化。我们在下面详细介绍它们。

2018年双11全球购物节期间（2018年11月11日），阿里巴巴数据库集群每秒处理高达49.1万笔销售交易，相当于每秒超过7000万笔数据库交易。为了经受住这一考验，我们引入了新的 OLTP 存储引擎 X-Engine，因为事务处理性能的一个重要部分归结为如何有效地使数据持久化并从存储中检索数据。

电子商务工作负载的主要挑战。我们首先确定三个关键技术挑战。

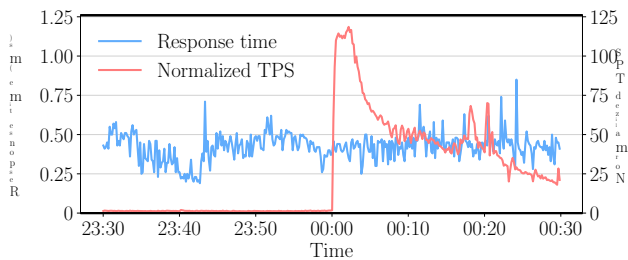


图 1: 2018 年双十一全球购物节期间, 运行 X-Engine 的在线数据库集群观察到每秒交易量突然增加 122 倍, 同时响应时间稳定。

海啸问题。阿里巴巴的电子商务平台通常需要在精心挑选的日期、全球客户普遍期望的、在整个市场规模上开展促销活动。例如, 每年 11 月 11 日举办的“双十一”全球购物节, 天猫上几乎所有卖家的大型促销活动都在午夜开始。尽管许多促销活动持续当天, 但也有一些大幅折扣仅在有限的时间内或数量有限 (先到先得)。从 11 月 11 日 00:00:00 开始, 这会导致底层存储引擎的事务工作负载 (反映为每秒事务量与时间的巨大垂直峰值) 急剧增加, 就像巨大的海啸袭击海岸一样。

图1描绘了2018年光棍节运行X-Engine的在线数据库集群的事务响应时间和每秒事务数 (TPS) (以11月11日午夜前一天的平均TPS标准化)。在午夜后的第一秒, 该集群的事务数急剧增加, 比前一秒高出约122倍。在X-Engine的帮助下, 成功保持了稳定的响应时间 (平均0.42 ms)。

为了应对图 1 所示的 122 次峰值, 阿里巴巴对 OLTP 数据库采用了无共享架构, 其中应用分片在多个数据库实例之间分配事务, 然后在峰值到来之前扩展数据库实例的数量。尽管这种方法有效, 但由于所需的实例数量巨大, 因此需要大量的资金和工程成本。在本文中, 我们通过提高OLTP数据库核心组件存储引擎的单机容量来解决这个问题, 从而减少给定峰值和吞吐量所需的实例数量, 或者增加在固定成本下可实现的吞吐量。

泄洪问题。存储引擎必须能够在处理高并发电子商务时快速将数据从主内存移动到持久存储 (即排出内存中积聚的洪水)

交易。在线促销会造成在线电子商务交易的高峰时段, 与非高峰时段相比, 其中的写入量 (例如下订单、更新库存和付款) 要高得多。尽管近年来主存容量稳步增长, 但与双十一期间大量交易插入或更新的记录大小相比, 仍然显得相形见绌。因此, 我们必须利用由 RAM、SSD 和 HDD 组成的内存层次结构中的可用容量。X-Engine通过采用分层存储布局来利用这种层次结构, 我们可以根据数据的温度 (即访问频率) 将数据放置在不同的层中, 并在某些层中使用NVM等新的内存技术。这很像通过逐级增加规模的水库排放流动的洪水。

LSM 树结构是分层存储的自然选择。除了 LSM 树之外, 加速写入的常见技术还包括基于日志的数据结构的仅附加方法 [27, 34]、优化的基于树的结构 [3, 18] 以及两者的混合形式 [31]。我们发现单独使用这些方法中的任何一种都不足以服务于此类电子商务交易: 有些假设列式存储不适合写入密集型交易; 其他人则通过牺牲点和范围查询的性能来提高写入的性能, 这不适合混合读写电子商务工作负载。LSM 树结构包含一个内存驻留组件, 我们可以在其中应用仅附加方法进行快速插入, 并且磁盘驻留组件包含组织在树中的多个包含级别, 每个级别明显大于其相邻的上层[15,16,27]。这种数据结构非常适合分层存储, 方便我们解决海啸和泄洪问题。

当前快速发展的问题。对于大多数数据库工作负载, 热记录通常在稳定的时间段内表现出很强的空间局部性。对于电子商务工作负载而言, 情况并非总是如此, 尤其是在双十一购物节等重大促销活动中。记录的空间局部性会随着时间的推移而快速变化。这是因为随着时间的推移, 不同类别或记录上有不同的促销活动。例如, 全天针对不同品类或品牌举办“秒杀”促销活动 (商品卖得火爆, 必须抓紧时间抢购), 刺激需求, 吸引顾客分时段购买不同商品。这意味着数据库缓存中的热记录会不断变化, 并且任何记录的温度都可能快速从冷/温变为热或从热变为冷/温。如果我们将数据库缓存视为水库, 将底层 (大型) 数据库视为海洋, 这种现象会导致当前 (即热记录与冷记录)

在非常深的海洋中（即存储在 X-Engine 上的巨大数据库）快速向任何方向移动。存储引擎需要确保能够尽快从深水中检索新出现的热记录并有效地缓存它们。

我们的贡献。我们在本文中介绍了 X-Engine。X-Engine 是一款基于 LSM-tree 的 OLTP 数据库存储引擎，旨在解决阿里巴巴电商平台面临的上述挑战。它通过利用多核处理器中的线程级并行性（TLP）来处理主存储器中的大多数请求，将写入与事务分离以使它们异步，并将长写入路径分解为管道中的多个阶段以提高整体吞吐量。为了解决洪水排放问题，X-Engine 采用分层存储方法，利用精细的 LSM 树结构和优化的压缩算法在不同层之间移动记录。我们还在压缩上应用了 FPGA 卸载。最后，为了解决当前快速发展的数据问题，我们引入了多版本元数据索引，该索引以写时复制的方式更新，以加速分层存储中的点查找，而不管数据温度如何。总而言之，我们的贡献是：

- 我们确定了 OLTP 存储引擎在处理电子商务交易时面临的三个主要挑战，并设计了基于 LSM 树结构的 X-Engine。
- 我们引入了一系列优化来解决已发现的问题，例如精炼的 LSM 树数据结构、FPGA 加速压缩、事务中的异步写入、多阶段管道和多版本元数据索引。
- 我们使用标准基准和电子商务工作负载广泛评估 X-Engine。结果表明，X-Engine 在在线促销期间的电子商务工作负载方面优于 InnoDB 和 RocksDB。我们还表明，X-Engine 经受住了双十一购物节海啸的考验。

本文的结构如下。第 2 节给出了 X-Engine 的概述。第 3 节介绍了我们优化的详细设计。我们在第 4 节中通过与使用 MySQL 的 InnoDB 和 RocksDB 进行比较来评估我们的设计和优化。最后，我们在第 5 节中讨论相关工作并在第 6 节中进行总结。

2 系统概述

X-Engine 是基于细化 LSM-tree 结构的分层 OLTP 存储引擎。正如第 1 节中所讨论的，由于电子商务事务工作负载中的数据温度不断变化，因此我们有动机区分不同存储层中记录的放置，以分别服务于热记录、温记录和冷记录。这使得 X-Engine 可以根据需要定制设计数据结构和访问方法。

记录温度（即最近窗口中的访问频率）。此外，由于电子商务交易涉及大量的插入和更新，因此我们采用 LSM 树，因为它具有出色的写入性能[27]。然而，我们发现传统的 LSM 树不足以支持电子商务事务工作负载。因此，我们在 X-Engine 中设计并应用了一套优化来解决之前确定的三个挑战。X-Engine 可以作为 POLARDB 的一部分部署在 POLARFS 之上 [4]。POLARFS 利用了许多新兴技术来实现超低延迟的文件系统，例如 RDMA 和 NVMe。

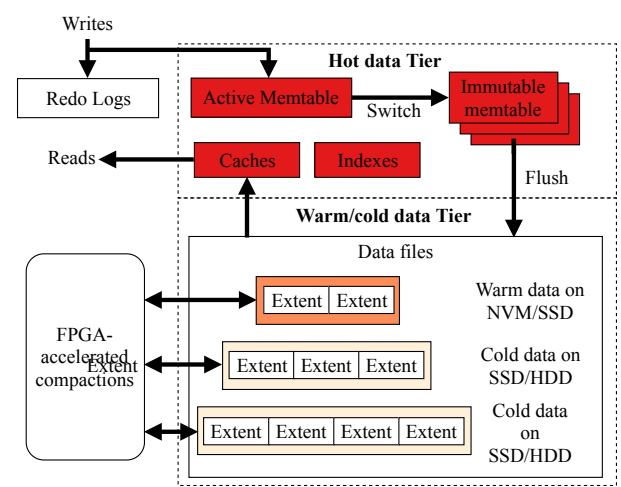


图2：X-Engine的架构。

存储布局。X-Engine的架构如图2所示。X-Engine 将每个表划分为多个子表，并维护一个 LSM 树、每个子表关联的元快照和索引。X-Engine 每个数据库实例包含一个重做日志。每个 LSM 树由驻留在主存中的热数据层和驻留在 NVM/SSD/HDD 中的温/冷数据层（进一步划分为不同的级别）组成，其中术语“热”、“温”和“冷”指的是数据温度，代表应放置在相应层中的数据的理想访问频率。热数据层包含一个活动的内存表和多个不可变的内存表，它们是存储最近插入的记录跳表的缓存，以及用于缓冲热记录的缓存。热/冷数据层以树状结构组织数据，树的每一层都存储排序的范围序列。扩展区打包记录块及其关联的过滤器和索引。我们正在探索机器学习技术来识别适当的数据温度，附录 C 对此进行了简要讨论，其完整细节超出了本工作的范围。

X-Engine 利用重做日志、元快照和索引来支持事务处理的多版本并发控制 (MVCC)。每个元快照都有一个元数据

表 1: X-Engine 中的优化摘要。

Optimization	Description	Problem
Asynchronous writes in transactions	Decoupling the commits of writes from the processing transactions.	Tsunami
Multi-staged pipeline	Decomposing the commits of writes to multiple pipeline stages.	
Fast flush in $Level_0$	Refining the $Level_0$ in the LSM-tree to accelerate flush.	Flood discharge
Data reuse	Reusing extents with non-overlapped key ranges in compactions.	
FPGA-accelerated compactions	Offloading compactions to FPGAs.	
Optimizing extents	Packaging data blocks, their corresponding filters, and indexes in extents.	Fast-moving current
Multi-version metadata index	Indexing all extents and memtables with versions for fast lookups.	
Caches	Buffering hot records using multiple kinds of caches.	
Incremental cache replacement	Replacing cached data incrementally during compactions.	

跟踪快照中树的所有级别中的所有内存表和范围的索引。树的一个或多个相邻级别形成一层，分别存储在 NV M、SSD 和 HDD 上。在 X-Engine 中，表被划分为多个子表。每个子表都有自己的热、暖和冷数据层（即 LSM 树）。X-Engine 以面向行的格式存储记录。我们设计了一个多版本 memtables 来存储不同版本的记录以支持 MV CC（在 3.2.1 节中介绍）。在磁盘上，元数据索引跟踪存储在盘区中的所有记录版本。我们在 3.1 节中介绍数据结构的细节。

读取路径。读取路径是从存储中检索记录的过程。原来的 LSM-tree 设计并没有很好的读性能。查找首先搜索内存表。如果内存表未命中，它必须逐一遍历每个连续的级别。在最坏的情况下，查找最终必须扫描所有级别，直到最大级别才能得出查询记录不存在的结论。为了加速这个过程，建议使用清单文件来定位包含查询密钥的目标 SST [9]。布隆过滤器也应用于每个 SST 中以促进提前终止。

为了对电子商务交易中常见的点查找实现良好的响应时间，我们优化了范围的设计，引入了跟踪所有内存表和范围的元数据索引，以及一组缓存以促进快速查找。我们还提出了压缩中的增量缓存替换方法，以减少压缩引起的不必要的缓存驱逐。我们引入快照来确保查询读取正确版本的记录。

写入路径。写入路径包括物理访问路径以及在存储引擎中插入或更新记录的相关过程。在 LSM 树 KV 存储中，到达的键值对被附加或插入到活动内存表中。一旦完全填满，活动内存表就会切换为不可变状态，等待刷新到磁盘。同时，创建一个新的空活动内存表。为了支持高并发的事务处理，存储引擎需要通过将新记录记录到持久存储（例如 SSD）中来使新记录持久化，并将它们高速插入到内存表中。我们在此过程中区分长延迟磁盘 I/O 和低延迟内存访问，并

将它们组织在多级管道中，以减少每个线程的空闲状态并提高整体吞吐量（第 3.2.3 节）。为了实现高并发性，我们进一步将写入的提交与事务的处理解耦，并分别优化它们的线程级并行性。

冲洗并压实。LSM 树依靠刷新和压缩操作来合并可能会淹没主内存的数据，从内存表到磁盘，并保持合并后的数据按排序顺序排列。不可变内存表被刷新到 $Level_0$ ，在此期间记录被排序并打包在排序序列表（SST）中 [16]。每个 SST 占用一个专有的键范围，因此一个级别可能包含多个 SST。当 $Level_i$ 中的 SST 大小达到阈值时，它们将与 $Level_{i+1}$ 中具有重叠键范围的 SST 合并。在某些系统中，此合并过程称为压缩，因为它还会删除标记为要删除的记录。原始压缩算法从两个级别读取 SST，合并它们，然后将合并结果写回 LSM 树。这个过程有几个缺点：它消耗大量的 CPU 和磁盘 I/O；同一条记录被多次读写 LSM-tree，造成写放大；它会使正在合并的记录的缓存内容无效，即使它们的值保持不变。

在 X-Engine 中，我们首先优化了不可变内存表的刷新。对于压缩，我们应用数据重用来减少要合并的范围数量，异步 I/O 与磁盘 I/O 重叠合并，以及 FPGA 卸载来减少 CPU 消耗。

优化总结。表 1 总结了针对其旨在解决的特定问题的各种优化。我们在下面的章节中介绍 X-Engine 的详细设计。

3 详细设计

在本节中，我们首先阐述 X-Engine 如何处理事务，然后介绍 X-Engine 关键组件的详细设计，如第 2 节所示，包括读路径、写路径、刷新和压缩。

X-Engine 应用 MVCC 和 2PL（两相锁定）实现 SI（快照隔离）和 RC 的隔离级别

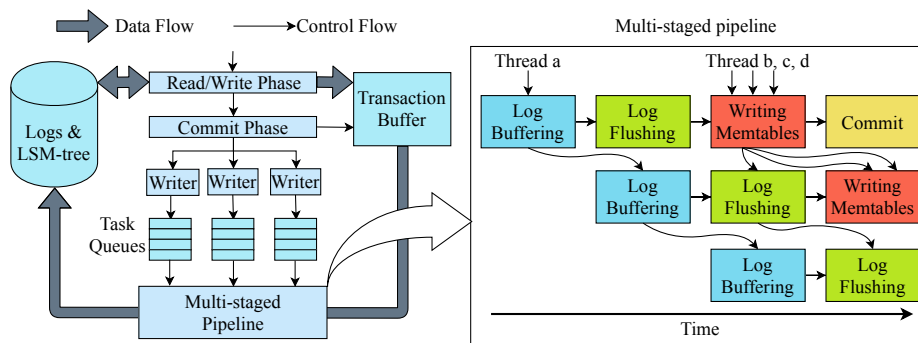


图3：X-Engine中事务处理的详细设计。

（读已提交）以保证事务的 ACID 属性。同一记录的不同版本存储为具有自动增量版本 ID 的单独元组。X-Engine 跟踪这些版本中的最大值作为 LSN（日志序列号）。每个传入事务都使用它视为快照的 LSN。事务只读取最大版本小于自己 LSN 的元组，并为其写入的每个元组添加行锁以避免写冲突。

图 3 显示了 X-Engine 中处理事务的概述。该过程由读/写阶段和提交阶段组成。事务的所有读请求都在读/写阶段通过访问 LSM 树的读路径得到服务。在此阶段，事务中要插入或更新的记录被写入事务缓冲区。接下来，在提交阶段，将事务缓冲区中的记录写入存储的任务被分发到多个写入任务队列。引入多阶段管道来通过记录相应的记录并将其插入到 LSM 树中来处理所有这些写入任务。下面详细介绍 X-Engine 的数据结构、读路径、写路径、flush 和 compaction。

3.1 读取路径

我们从数据结构的设计开始，包括范围、缓存和索引。对于每种数据结构，我们介绍它如何促进读取路径中的快速查找。

3.1.1 盘区 图 4 显示了盘区的布局，由数据块、模式数据和块索引组成。记录以面向行的方式存储在数据块中。架构数据跟踪每列的类型。块索引保存每个数据块的偏移量。在我们生产系统的当前部署中，我们将 LSM 树所有级别的范围总大小调整为 2 MB。由于许多电子商务交易以高度倾斜的方式访问记录，因此将范围保持在这个大小允许在压缩期间重用许多范围（更多详细信息请参见第 3.3.2 节）。这种设计还有助于压缩期间的增量缓存替换（第 3.1.4 节中介绍的优化）。

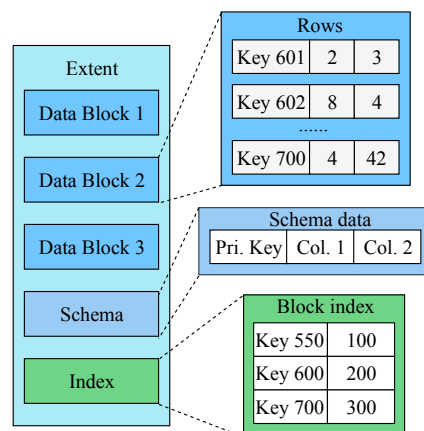


图 4：范围的布局。

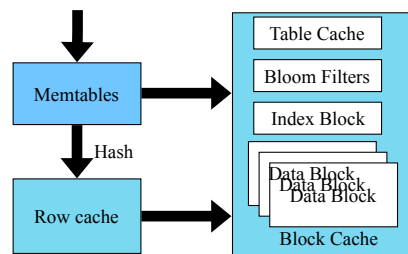


图 5：缓存的结构。

我们在每个范围内存储具有版本的模式数据，以加速 DDL（数据定义语言）操作。通过这种设计，当向表中添加新列时，我们只需要在新版本的新范围上强制执行此新列，而无需修改任何现有的。当查询读取具有不同版本模式的盘区时，它会与最新版本一致，并将默认值填充到具有旧模式的记录的空属性中。这种快速 DDL 功能对于在线电子商务企业非常重要，因为它们经常调整数据库模式的设计以适应需求的变化。

3.1.2 缓存 图5 展示了X-Engine 中的数据库缓存。我们专门针对点查找（阿里巴巴电子商务交易中的大部分查询）优化了行缓存。行缓存使用以下方式缓冲记录

LRU 缓存替换策略，无论记录驻留在 LSM 树中的哪个级别。因此，即使是最大级别的记录，只要查询访问它就可以被缓存。一旦点查找错过内存表，查询的键就会被散列到行缓存中相应的槽以进行匹配。因此，在行缓存中检索记录以进行点查询只需要 $O(1)$ 时间。当查找随机访问记录时，行缓存的影响较小。

我们只在行缓存中保留最新版本的记录，由于时间局部性，这些记录有最大的机会被访问。为了实现这一点，我们在刷新期间用行缓存中的新记录替换旧版本的记录，从而减少可能因刷新而导致的缓存未命中。

块缓存以数据块为单位缓冲数据。它为每个错过行缓存的请求或范围查询的查找提供服务。表缓存包含通向相应盘区的子表头的元数据信息。找到范围后，我们使用布隆过滤器来过滤掉不匹配的键。然后，我们搜索索引块来定位该记录，最后从其数据块中检索它。

这些缓存对于减少记录温度变化后的缓存未命中非常重要。由于记录的空间局部性，新出现的热点记录和行缓存中现有的热点记录可能来自相同的盘区甚至相同的数据块。因此，表和块缓存有助于提高缓存未命中后的总体缓存命中率，并且可能有助于减少行缓存中替换的延迟。

3.1.3 多版本元数据索引 X-Engine中多版本元数据索引的结构如图6所示。每个子表的LSM树都有其关联的元数据索引，该索引从代表子表的根节点开始。索引的每次修改都会创建一个新的元快照，该元快照指向所有关联的级别和内存表，而无需修改现有元快照的节点（即写时复制方法）。在图 6 中， $extent_i$ 最初是 $Level_0$ 的一部分并被缓存（以红色显示）。当重用此范围的压缩完成时， $MetaSnapshot_v$ 一侧会创建一个新的 $MetaSnapshot_{v+1}$ ，链接到新合并的颜色较深的 $Level_1$ （颜色）。 $Level_1$ 的元数据只需要指向 $extent_i$ ，而不需要实际在磁盘上移动它（紫色），保持其所有缓存内容完好无损。利用这种写时复制方法，事务可以以只读方式访问它们想要的任何版本，而无需在数据访问期间锁定索引。我们应用垃圾收集来删除过时的元快照。其他存储引擎（例如 RocksDB [14]）也探索了类似的设计。

3.1.4 增量缓存替换 在LSM树中，由于压缩合并了磁盘中的许多范围，因此通常会导致大批量的缓存驱逐，从而减少缓存

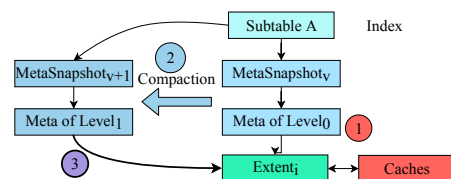


图 6：以写时复制方式更新元数据索引的示例。

查找命中率并导致明显的性能下降和不稳定的响应时间。即使缓存记录的值没有改变，如果它们的范围与压缩中涉及的其他范围共享重叠的键范围，它们也可能已在磁盘中移动。

为了解决这个问题，我们建议在块缓存中进行增量替换，而不是从缓存中逐出所有压缩范围。在压缩过程中，我们检查要合并的范围的数据块是否被缓存。如果是这样，我们将在同一位置用新合并的块替换缓存中的旧块，而不是简单地逐出所有旧块。这种方法通过在块高速缓存中保留一些已更新和未移动的块来减少高速缓存未命中。

3.2 写入路径

在本节中，我们首先优化 memtable 结构，该结构接收每个子表的 LSM 树的传入记录。接下来，我们介绍如何设计写任务队列和写路径中的多级管道，这些队列是由 X-Engine 中所有子表的 LSM-tree 共享的。

3.2.1 多版本内存表 我们像许多其他系统一样将内存表实现为无锁跳表，以实现良好的查找和插入性能[9]。然而，基于跳表的内存表的最先进实现在查询热记录时存在性能问题。对单个记录的频繁更新会生成多个版本。如果热门记录与仅对最新版本感兴趣的查询的谓词相匹配，则查询可能必须扫描许多旧版本才能找到所请求的版本。当顾客下单购买热门商品时，电子商务平台的在线促销活动会放大这些冗余访问。

在X-Engine中，我们将同一记录的新版本垂直附加到原始节点旁边，形成一个新的链表。图 7 显示了建议的结构，其中蓝色节点存储具有不同键的记录，黄色节点存储同一记录的多个版本。不同的密钥被组织在一个跳跃列表中，而每个密钥的多个版本存储在一个链接列表中。热点记录的更新会增加其相应的链表。此外，通常由传入事务引用的新版本保持在靠近唯一键跳跃列表底部的位置，如图 7 所示，其中版本 99 是最新版本。这个设计

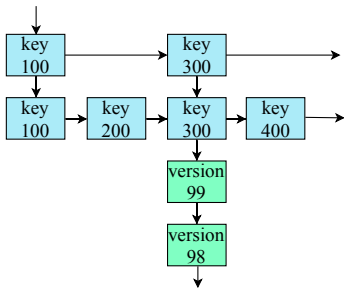


图 7: 多版本内存表的结构。减少扫描不必要的旧版本所造成的开销。

3.2.2 事务异步写入 InnoDB 等存储引擎中传统的一线程一事务方式在写入效率上存在明显缺陷。在这种方法中，用户线程从头到尾执行事务。虽然这种方法可以很容易地实现写入的执行和事务的并发控制，而用户线程本身并不写日志，但该线程必须等待写入日志的磁盘IO的较长延迟才能完成。

在X-Engine中，我们选择了另一种方法来将写入的提交与其相应的事务解耦，并将它们分组以进行批处理。如图3所示，我们首先将写任务分配到多个无锁写任务队列中。此后，大多数线程可以异步返回来处理其他事务，每个队列只留下一个线程参与提交多阶段管道中的写入任务（下面将介绍）。通过这种方式，事务中的写入是异步的。在高度并发的负载中，此方法允许更多线程处理并发事务中的写入任务。队列的最佳数量受到机器中可用的 I/O 带宽以及多个线程之间对每个无锁队列头部的争用的限制。我们发现每个队列 8 个线程会使 32 核机器中的 I/O 带宽饱和（第 4.1 节），并且为每个队列分配更多线程会降低由于争用而导致的吞吐量。此外，在这个解耦过程之后，我们将同一队列中的写入任务分组在一起并批量处理它们。与单个事务提交相比，批量提交可以显着提高 I/O，从而提高吞吐量。我们在下面的讨论中优化批处理的效率。

3.2.3 多级管道 写入路径是访问主内存和磁盘的多个操作的长序列，并且随着执行的计算工作量不断变化。这使得通过计算隐藏内存访问和磁盘 I/O 变得具有挑战性。

为了解决这个问题，我们将写入路径分解为多个阶段。图 3 右半部分显示了概览

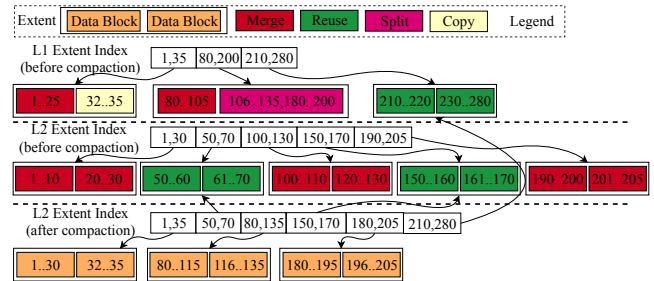


图 8: 压缩期间数据重用的示例。

四级管道的结构，其中各个阶段交替访问主内存和磁盘。在第一阶段，日志缓冲，线程将每个写请求的WAL（预写日志）从事务缓冲区收集到内存驻留日志缓冲区，并计算其相应的CRC32错误检测代码。此阶段涉及大量计算并且仅访问主存储器。第二阶段，日志刷新，线程将缓冲区中的日志刷新到磁盘。刷新日志后，日志序列号在日志文件中前进。然后，这些线程将已登录的写入任务推送到下一阶段，写入内存表。这里，多个线程并行地在活动内存表中追加记录。此阶段仅访问主存储器。所有此类写入都可以在失败后从 WAL 中恢复。在最后一个阶段，提交，所有任务完成的事务最终由多个线程并行提交，并释放它们使用的资源，例如锁。

在此管道中，我们根据每个阶段的要求分别调度线程，以使每个阶段的吞吐量与其他阶段相匹配，从而最大化总吞吐量。尽管前三个阶段是内存密集型的，但第一和第二阶段访问主存储器中的不同数据结构，而第二阶段写入磁盘。因此，重叠它们可以提高主内存和磁盘的利用率。

此外，我们分别限制每个阶段的线程数量。由于前两个阶段的数据依赖性很强，我们只为每个阶段调度一个线程（例如图 3 中的线程“a”）。对于其他阶段，我们分配多个线程进行并行处理（例如图 3 中的线程 b、c、d）。所有线程从阶段中提取任务进行处理。从前两个阶段拉取任务是抢占式操作，只允许第一个到达的线程处理该阶段。其他阶段是高度并行的，允许多个线程并行工作。

3.3 冲洗和压实

接下来，我们介绍如何优化X-Engine分层存储中组织记录的flush和compaction操作。

3.3.1 Level₀中暖区的快速刷新 我们依靠刷新来避免X-Engine中的内存不足故障，风险

这在交易高峰时非常重要。在 X-engine 中，每个刷新操作都会将其不可变内存表转换为范围，将它们附加到 $Level_0$ 并离开，而不将它们与现有记录合并。然而，这个过程留下了一组未排序的范围。查询现在必须访问所有范围才能找到潜在的匹配项。此过程中涉及的磁盘 I/O 成本很高。尽管 $Level_0$ 的大小可能小于整个存储的 1%，但它包含的记录仅比内存表中最近插入的记录稍旧。由于电子商务工作负载中具有很强的时间局部性，传入查询很可能需要这些记录。因此，我们将 $Level_0$ 中的范围称为暖范围。

我们引入了 $Level_0$ 内压缩来主动合并 $Level_0$ 中的热范围，而不将合并的范围推送到下一个 $Level_1$ 。这种方法将热记录保留在 LSM 树的第一层中，防止查询在树中深入以检索这些记录。另一方面，由于与其他级别相比， $Level_0$ 的大小较小，因此 $Level_0$ 内压缩只需要访问一小部分范围，这与其他在更深级别中广泛合并范围的压缩不同。由于 CPU 和 I/O 的轻量消耗，因此可以频繁执行级别内压缩。

3.3.2 加速压缩压缩涉及昂贵的合并操作。我们对压缩应用了三种优化：数据重用、异步 I/O 和 FPGA 卸载。

数据重用。我们在压缩过程中尽可能重用范围和数据块，以减少两个相邻级别（即 $Level_i$ 和 $Level_{i+1}$ ）之间合并所需的 I/O 数量。为了增加重用的机会并使其有效，我们将 extent 的大小减少到 2MB，并将 extent 进一步划分为多个 16KB 的数据块。如果压缩中涉及的某个盘区的键范围与其他盘区的键范围不重叠，我们可以通过简单地更新其相应的元数据索引（如第 3.1.3 节中介绍的）来重用它们，而无需实际在磁盘上移动它们。我们在图 8 中展示了一个示例，其中 $Level_1$ 中键范围为 [1, 35]、[80, 200] 和 [210, 280] 的三个范围将被压缩为 $Level_2$ 中键范围为 [1, 30] 的 5 个范围，[50, 70]、[100, 130]、[150, 170] 和 [190, 205]。我们列出了不同的重用案例如下：

- $Level_1$ 中的范围 [210, 280]， $Level_2$ 中的 [50, 70] 直接重用。
- 级别₁范围[1, 35]与级别₂范围[1, 30]重叠。然而，前者中只有一个数据块[1, 25]与后者中的数据块重叠。因此，数据块[32, 35]被重用。
- $Level_1$ 范围[80, 200]与 $Level_2$ 中的多个范围重叠。它的第二个数据块与 $Level_2$ 中的三个范围重叠。然而，这个数据块中的键

是稀疏的，因为 135 和 180 之间没有键。因此，我们将其分成两个数据块：[106, 135] 和 [180, 200]，并将它们与范围 [100, 130] 和 [190, 205] 合并分别为₂级别。直接重用范围[150, 170]。

异步 I/O。在范围级别，压缩操作由三个不相交的阶段组成：（1）从存储中检索两个输入范围，（2）合并它们，以及（3）将合并的范围（一个或多个）写回存储。第一和第三阶段是 I/O 阶段，而第二阶段是计算密集型阶段。我们在第一和第三阶段发出异步 I/O 请求。第二阶段作为第一 I/O 阶段的回调函数来实现。当多个压缩并行运行时，第二阶段的执行与其他阶段的执行重叠以隐藏 I/O。

FPGA 卸载。我们引入 FPGA 来加速压缩，并减少 CPU 上的资源消耗。通过上面介绍的两种优化，在 CPU 上运行的压缩仍然会消耗多个线程。因为压缩工作在来自 LSM 树的两个连续级别的独立扩展对上，所以压缩任务在这些对的粒度上是令人尴尬的并行。因此，它可以分成多个小任务。我们将这样的小任务卸载到 FPGA，并以流式处理方式处理它们。每个压缩范围都会传输回磁盘。通过这种卸载，CPU 线程可以从合并范围的沉重负担中解放出来。因此，我们能够分配更多线程来处理并发事务。这种基于 FPGA 的压缩加速的细节超出了本文的范围，将单独探讨。

3.3.3 调度压缩 LSM 树依靠压缩来保持记录的排序顺序并删除标记为从存储中删除的记录。如果不及时删除已删除的记录，查找查询可能必须遍历存储中的许多无效记录，从而严重损害读取性能。压缩还有助于合并 $Level_0$ 中的子级别，以减少该级别的查找成本。

在 X-Engine 中，我们引入了基于规则的压缩调度来利用这些好处。我们根据压缩操作的级别来区分压缩：级别内₀压缩（合并级别₀内的子级别），次要压缩（合并除最大级别之外的两个相邻级别），主要压缩（合并最大级别及其之上的级别）和自主主要压缩（在最大级别内合并以减少碎片并删除已删除的记录）。当某个级别的总大小或范围总数达到预定义的阈值时，会触发压缩。所有触发的压缩作业都会排队到优先级队列中。确定优先级的规则是开放配置的，这进一步取决于数据库之上不同应用程序的要求。

下面，我们展示了为阿里巴巴的一个在线应用程序量身定制的配置示例。在这个应用程序中，存在频繁的删除。当需要删除的记录数量达到阈值时，会触发一次compaction来删除这些记录，并且该compaction被赋予最高优先级，以防止存储空间的浪费。删除后，会优先进行Level₀内压缩，以帮助加速在Level₀中查找最近插入的记录。小压缩、主要压缩和自主压缩分别按以下顺序确定优先级：

- (1) 删除压缩。(2) 层内₀压缩
- 。(3) 轻微压实。(4) 大压实
- 。(5) 自主压缩。

不同子表以及不同LSM树的压缩可以安排并发执行。由于子表之间的数据独立性，这种并行性是令人尴尬的。在每个子表内，一次仅执行一次压缩。虽然可以对同一LSM树执行多次压缩而不会损坏数据或导致任何数据争用，但我们有足够的机会在子表级别进行并发压缩以获得性能优势。

4 评价

在本节中，我们首先展示X-Engine在处理混合不同类型查询的电子商务OLTP工作负载时的表现。然后，我们评估了X-Engine在解决海啸问题、泄洪问题和快速流动问题这三个已确定的挑战方面的贡献。我们已经测量了双十一期间运行X-Engine的MySQL数据库的实际性能，如图1所示。

4.1 实验装置

我们将X-Engine与其他两种流行的存储引擎进行比较：InnoDB [26] 和 RocksDB [15]。InnoDB is the default storage engine of MySQL. RocksDB 是一种广泛使用的 LSM 树存储引擎，从 LevelDB [16] 发展而来。在阿里巴巴，我们在许多数据库集群中使用 InnoDB 和 X-Engine 以及 MySQL 5.7 来处理电子商务交易。因此，MySQL 5.7 是评估 InnoDB 和 X-Engine 的自然选择。我们使用最新版本（2018 年 11 月）的 RocksDB [14] 和 MyRocks (Rock sDB 上的 MySQL) [11]。

正如第 1 节中所讨论的，我们在生产中对多个数据库实例应用分片以进行分布式处理，分片的细节不是本文的重点。我们在单个节点设置（即目标机器）中部署每个存储引擎及其关联的 MySQL 实例。目标机器由两个 16 核 Intel E5-2682 处理器（总共 64 个硬件线程）、一个 512 GB Samsung DDR4-2133 组成

主内存和由三个 Intel SSD 组成的 RAID。操作系统是Linux 4.9.79。我们使用另一台具有相同硬件配置的机器（即客户端机器）来向目标机器发出 SQL 查询。这两台机器位于同一数据中心，它们之间的网络延迟约为 0.07 毫秒，该延迟已包含在所有测量中。

除了SysBench和dbbench等流行的基准工具包之外，我们还应用了X-Bench（阿里巴巴为压力测试而开发的基准工具包）来综合由点查找、范围查找、更新和插入组成的混合电子商务工作负载。在阿里巴巴数据库管理员和在线运行电子商务业务的数据库用户的帮助下，我们能够通过调整这四种查询类型的混合比例来密切模拟现实世界的电子商务工作负载。有关 X-Bench 的更多详细信息请参见附录 B。

请注意，对于所有箱线图，我们使用 25% 和 75% 分位数作为箱边界，中值作为箱中的黄线，最小和最大值作为外部边界（水平线）。

4.2 电子商务交易

图 9 显示了处理电子商务工作负载时的性能。我们从 80 % 点查找、11% 范围查找、6% 更新和 3% 插入的混合开始（在图中表示为 80:11:6:3）。这种混合最类似于阿里巴巴许多数据库中的读取密集型工作负载，而没有受到在线促销的影响。我们逐渐将读取（点查找和范围查找）的份额缩小到 42% 和 10%，同时分别保持相同的更新和插入百分比。这个 42:10:32:16 的混合时间非常接近我们在双十一等在线促销活动中观察到的混合时间。对于读取密集型的混合，InnoDB 比 RocksDB 和 X-Engine 更快、更稳定。在 LSM 树系统中，主存中缺少内存表和缓存的查找会因访问磁盘中的一个或多个级别而受到影响，这会降低响应时间并增加方差。通过对读取路径进行一系列优化，在这些读取密集型情况下，X-Engine 的性能仅比 InnoDB 稍差（<10%）。在双十一工作负载的关键42:10:32:16案例中，X-Engine在QPS方面表现非常稳定，平均比InnoDB和RocksDB分别高出44%和31%。

接下来，我们评估优化的影响，帮助X-Engine处理具有大量写入的高并发电子商务交易。

4.3 海啸问题

回想一下，解决海啸问题意味着提高存储引擎的峰值吞吐量。在

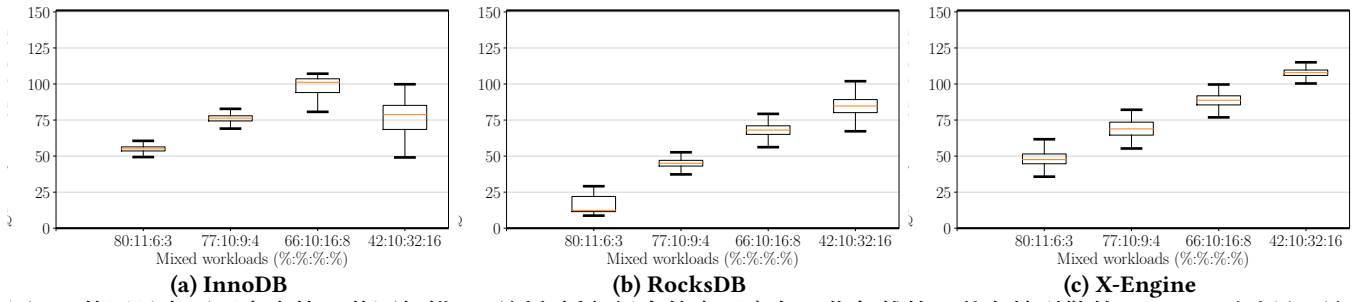


图 9: 使用具有不同点查找、范围扫描、更新和插入组合的电子商务工作负载的三种存储引擎的 MySQL 吞吐量。这些图的 y 轴是相同的。

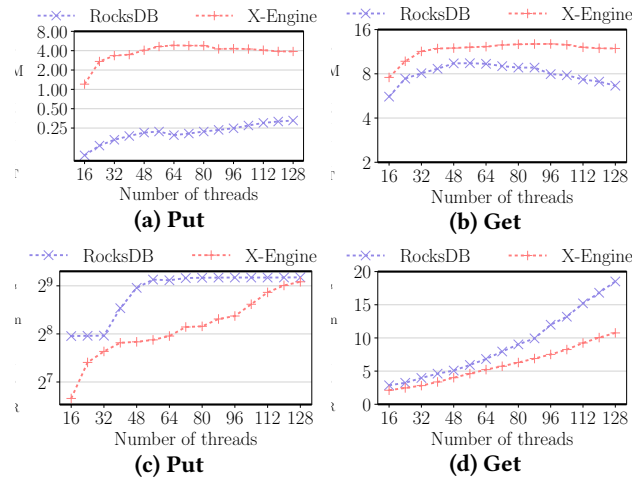


Figure 10: Throughput and response time of storage engine through the key-value interface.

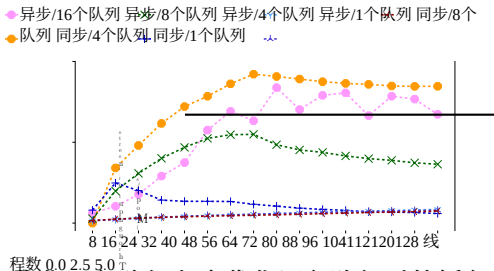


图 11: 优化写入路径和未优化写入路径时的插入吞吐量。

在本节中，我们使用dbbench [25]直接通过键值 (KV) 接口评估存储引擎，并使用SysBench [20]通过SQL接口评估其相应的MySQL实例。在这些评估中，每个KV对都有16字节的密钥和10字节的值；每条记录包含一个12字节的密钥和500字节的属性。这些尺寸在电子商务交易中很常见。我们在KV评估中忽略了InnoDB，因为它没有KV接口。

4.3.1 在没有MySQL的情况下运行KV操作 如图10所示，我们通过将软件线程数量增加到128个来对存储引擎的KV接口进行压力。对于put，X-Engine的吞吐量提高了23倍

与RocksDB相比，响应时间相当。对于get来说，X-Engine在吞吐量和响应时间方面分别比RocksDB快1.68倍和1.67倍。

为了解释这些结果，我们在图11中进一步展示了异步写入、写入任务队列和多级管道（在3.2.2节中介绍）的影响。在相同的8个写入任务队列的情况下，异步写入的X-Engine峰值比同步写入快11倍。通过将写入任务队列的数量从1个增加到8个，X-Engine的速度提高了4倍。请注意，仅使用一个任务队列会禁用多阶段管道，因为管道中只有一个线程，因此任何阶段都无法并行执行。当使用所有硬件线程时，实现了最高吞吐量，这表明X-Engine有效地利用了线程级并行性。

4.3.2 使用MySQL运行SQL查询 图12和图13分别绘制了三个存储引擎通过SQL接口测得的吞吐量和响应时间。在这些实验中，我们将连接数从1扩展到512。在吞吐量方面，X-Engine的点查找、插入和更新查询分别比第二佳快1.60、4.25和2.99倍。随着连接数量的增加，X-Engine的吞吐量和响应时间也得到了更好的扩展，如图11所示，其提高了线程级效率。

尽管范围查找在阿里巴巴的电商工作负载中只占一小部分查询，但它是在线数据库的必要功能。图14显示了三个存储引擎中扫描的记录数量不同的范围查找的吞吐量。X-Engine和RocksDB的性能都比InnoDB差，因为它们LSM-tree存储布局对扫描不友好。我们正在进行的工作是在X-Engine中采用扫描优化。

4.4 泄洪问题

X-Engine通过提高压实性能来促进洪水的排放。图15比较了运行SysBench的oltp_insert时X-Engine是否将其卸载到FPGA的吞吐量。FPGA将吞吐量提高了27%，并减少了差异。我们在附录A中展示了通过FPGA卸载节省的CPU。

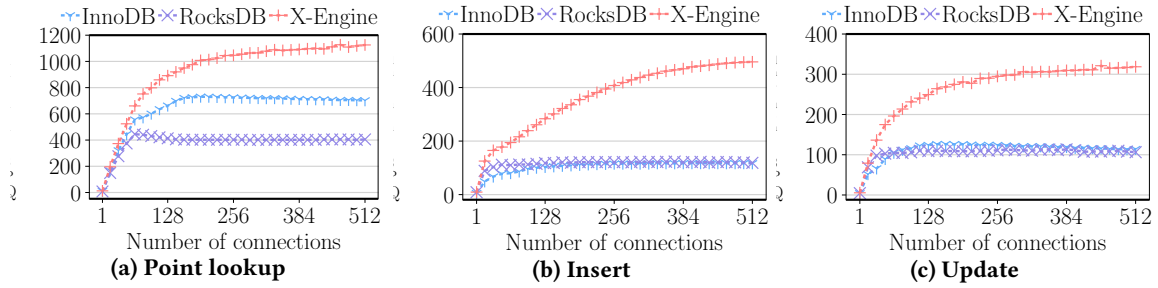


Figure 12: Throughput of storage engines through SQL interfaces.

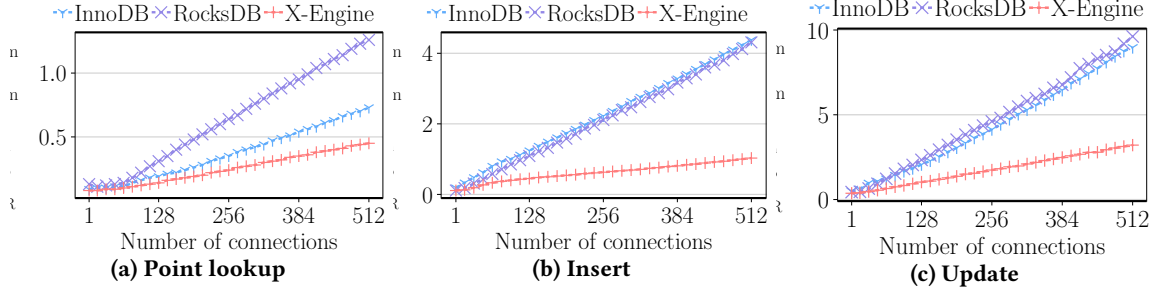


Figure 13: Response time of storage engines through SQL interfaces.

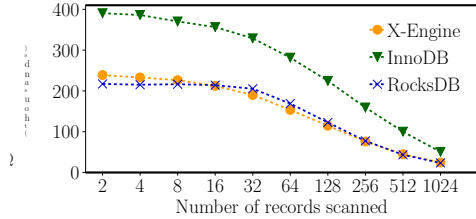


Figure 14: Throughput of range lookups.

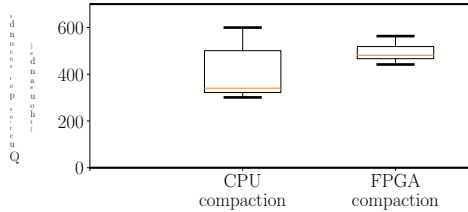


图 15: 使用和不使用用于压缩的 FPGA 卸载的 MySQL (X-Engine) 吞吐量。

为了评估X-Engine在压缩过程中数据重用的效率，我们准备了两组记录，分别包含5000万条和5亿条记录。所有密钥均随机均匀分布。我们将5000万次运行中不同密钥的百分比从70%更改为99%。其余部分包含具有相同密钥的同一记录的不同版本的组。图16显示了压缩这两次运行时 RocksDB 和 X-Engine 的吞吐量。当90%和99%的键是不同的时，X-Engine的速度分别快2.99倍和19.83倍。在这些情况下，具有相同密钥的不同版本的记录的分布是高度倾斜的，类似于我们在具有热点的电子商务工作负载中观察到的情况。保持较小的盘区和数据块（分别为2MB和16KB）会增加数量

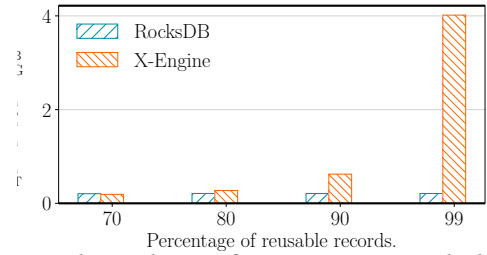
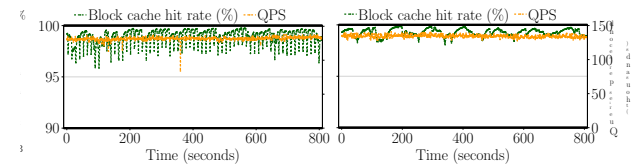


Figure 16: Throughput of compactions with different percentages of distinct records.



(a) 没有增量缓存替换。(b) 增量缓存替换。图 17: X-Engine 在处理电子商务工作负载时的块缓存命中率。这两个图共享相同的轴。

可重用范围并有助于减少压缩开销。

图17显示了X-Engine在处理电子商务工作负载（具有大量写入的42:10:32:16情况）时使用和不使用增量缓存替换的块缓存命中率。这种优化显著减少了块缓存命中率的变化，并避免了QPS偶尔突然下降（下降约36%）。我们承认，压缩和刷新等操作仍然会定期导致缓存驱逐并损害缓存命中率。

4.5 快速流动的电流问题

当前快速移动问题的解决方案取决于缓存的效率，这决定了我们访问冷记录（可能存储在磁盘上）并将其缓存在缓存中的速度。在图 18 中，我们根据 Zipf 分布改变点查询访问的键的偏度，并测量行和块缓存命中率以及实现的 QPS。当访问随机均匀分布时，几乎所有访问都会错过行缓存，因为将来的查询可能很难访问缓存的记录。随着更多的倾斜访问，一些记录变得很热，并被缓存在行缓存中，从而增加了行缓存命中率。这些结果表明，行缓存在电子商务工作负载中常见的高度倾斜场景中表现良好。然而，块缓存对于点查询的影响较小。

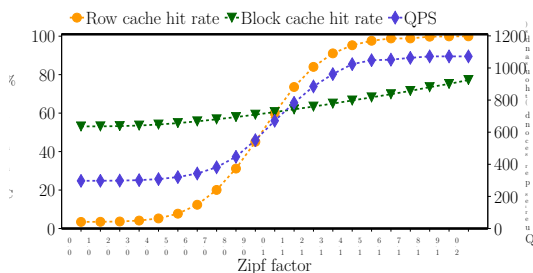


图 18：行缓存和块缓存对倾斜查找的影响。

5 相关工作

存储引擎是任何 OLTP 数据库系统的关键组件。Amazon Aurora 存储引擎是从 InnoDB 的一个分支开发出来的，它执行并行和异步写入，以减少延迟并提高写入吞吐量 [1, 33]。我们通过 X-Engine 中的多级管道改进了这一设计原理。利用基于日志的结构是优化写入性能的另一种有效方法[23,24,30]。LSM树是由O'Neil等人提出的。[27]，并吸引了许多后续努力来优化其设计[8,19,31]。钱德拉穆利等人。提出了一种具有混合日志的高度缓存优化的并发哈希索引，支持快速就地更新[5]。董等人。通过仔细研究相关权衡，优化了 RocksDB 中的空间放大 [9]。RocksDB 还包含一个单独的 WAL 写入阶段 [13]，以及内存表中的并发写入 [12]。在 X-Engine 中，我们采用了 LevelDB [16] 中 memtables 的思想，并提出了一系列优化来实现高并发的快速写入，从而显著减少 LSM 树的写入放大。

许多研究优化了 LSM 树系统中记录的合并方式。贾加迪什等人。将一个级别划分为多个子级别，并在压缩期间将它们合并到下一个级别中[17]。西尔斯等人。探索了 LSM 树每个级别的合并以稳定的方式逐步进行，减少了它们对读取性能的负面影响[31]。

拉朱等人。建议仅对警卫内部的记录进行排序，并允许警卫外部未排序的命令[28]。滕等人。引入了压缩缓冲区来维护频繁访问的数据，减少由压缩引起的缓存驱逐[32]。任等人。在基于图的系统等场景中重新设计了半排序数据的数据块索引[29]。达扬等人。对分层和平整压缩的成本进行了建模，并改进了更新成本和点查找/空间成本之间的权衡[8]。X-Engine 通过广泛的数据重用减少压缩的开销，避免大量不必要的合并，并建议将压缩卸载到 FPGA 以实现硬件辅助加速。

为了实现高效的读取操作，LSM 树中优化了多种数据结构，包括索引[6,15,21,22]，缓存[2,10,23]和过滤器[7]。X-Engine 将所有这些数据结构以分层的方式实践，并将索引扩展到各个范围。

6 结论

我们推出了 X-Engine，一种 OLTP 存储引擎，针对阿里巴巴全球最大的电子商务平台进行了优化，为全球超过 6 亿活跃客户提供服务。在线电子商务交易带来了一些独特的挑战，尤其是在一年一度的双十一全球购物节期间，顾客在短时间内进行大量购物。我们在优化的 LSM 树之上设计 X-Engine，以利用硬件加速（例如 FPGA 加速压缩）以及一系列优化（例如事务中的异步写入、多级管道、增量缓存替换和多种缓存）。X-Engine 在处理在线促销期间的电子商务工作负载时表现优于其他存储引擎，并成功服务于双十一购物节。我们正在进行和未来的工作包括采用共享存储设计来提高分片下无共享分布式事务处理的可扩展性，并应用机器学习方法来预测数据温度，以促进分层存储中记录放置的智能调度。

致谢

我们感谢审稿人对本文的反馈。X-Engine 由阿里巴巴集团的工程师团队设计、开发和管理。除了本文的作者之外，我们还感谢 Chang Cai、Shiping Chen、Zhi Kong、Canfang Shang、Dehao Wang、Lujun Wang、Buwen Wu、Fei Wu、Anan Zhao、Teng Zhang、Dongsheng Zhao 和 Yananzhi 设计和开发 X-Engine。我们非常感谢陈荣耀、陈文涛、李胜涛、李悦、王嘉怡和张锐对 X-Engine 进行在线评估和管理，作为他们支持阿里巴巴在线业务（包括一年一度的“双十一”购物节）的一部分。

参考

[1] 史蒂夫·亚伯拉罕。2018年。推出 Aurora 存储引擎。https://aws.amazon.com/cn/blogs/database/介绍-the-aurora-storage-engine/。[2] Mehmet Altinel, Christof Bornhövd, Sailesh Krishnamurthy, C. Mohan, Hamid Pirahesh 和 Berthold Reinwald。2003年。缓存表：为自适应数据库缓存铺平道路。第29届国际超大型数据库会议 (VLDB '03) 论文集, 卷。29。VLDB 捐赠基金, 718–729。[3] Michael A. Bender, Martin Farach-Colton, Jeremy T. Fineman, Yonatan R. Fogel, Bradley C. Kuszmaul 和 Jelani Nelson。2007。缓存无关的流式 B 树。第十九届 ACM 并行算法和架构年度研讨会 (SPAA '07) 论文集。ACM, 美国纽约州纽约市, 81–92。[4] 曹伟, 刘振军, 王鹏, 陈森, 朱彩凤, 郑松, 王宇辉, 马国庆。2018。PolarFS: 用于共享存储云数据库的超低延迟和故障恢复分布式文件系统。VLDB 捐赠论文集 11, 12 (2018), 1849–1862。[5] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter 和 Mike Barnett。2018。更快：具有就地更新的并发键值存储。2018 年国际数据管理会议 (SIGMOD '18) 会议记录。ACM, 美国纽约州纽约市, 275–290。[6] 陈世民, 菲利普·B·吉本斯, 托德·C·莫里。2001。通过预取提高索引性能。2001 年 ACM SIGMOD 国际数据管理会议 (SIGMOD '01) 会议记录。ACM, 美国纽约州纽约市, 235–246。[7] 尼夫·达扬 (Niv Dayan)、马诺斯·阿萨纳苏利斯 (Manos Athanassoulis) 和斯特拉托斯·伊德雷奥斯 (Stratos Idreos)。2018。LSM 树的最佳布隆过滤器和自适应合并。ACM 数据库系统交易 (2018)。[8] 尼夫·达扬和斯特拉托斯·伊德雷奥斯。2018。Do stoevsky: 通过自适应删除多余的合并, 为基于 LSM 树的键值存储提供更好的时空权衡。2018 年国际数据管理会议 (SIGMOD '18) 的会议记录。ACM, 美国纽约州纽约市, 505–520。[9] Siying Dong, Mark Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor 和 Michael Strum。2017。优化 RocksDB 中的空间放大。在每两年一次的创新数据系统研究会议 (CIDR) 中, 卷。3。[10] 克劳斯·埃尔哈特和鲁道夫·拜尔。1984。用于数据库系统中高性能和快速重启的数据库缓存。ACM 数据库系统交易 (TODS) 9, 4 (1984 年 12 月), 503–525。[11] 脸书。2018。我的摇滚。https://github.com/facebook/mysql-5.6/releases/tag/fb-prod201803。[12] 脸书。2018。RocksDB 内存表。https://github.com/facebook/rocksdb/wiki/MemTable。[13] 脸书。2018。RocksDB 流水线写入。https://github.com/facebook/rocksdb/wiki/Pipelined-Write。[14] 脸书。2018。RocksDB 版本 v5.17.2。https://github.com/facebook/rocksdb/releases/tag/v5.17.2。[15] 脸书。2019。RocksDB: 用于快速存储环境的持久键值存储。https://rocksdb.org/。[16] 桑杰·格马瓦特和杰夫·迪恩。2011。水平数据库。网址: s://github.com/google/leveldb,%20http://leveldb.组织 (2011)。[17] 戈茨·格雷夫和 Harumi Kuno。2010。自选择、自调整、增量优化索引。第13届扩展数据库技术国际会议 (EDBT '10) 的会议记录。ACM, 美国纽约州纽约市, 371–381。[18] Sándor Héman, Marcin Zukowski, Niels J. Nes, Lefteris Sidirourgos 和 Peter Boncz。2010。列存储中的位置更新处理。

2010 年 ACM SIGMOD 国际数据管理会议 (SIGMOD '10) 会议记录。ACM, 美国纽约, 543–554。[19] H. V. Jagadish, P. P. S. Narayan, S. Seshadri, S. Sudarshan 和 Rama Kanneganti。1997。数据记录和仓储的增量组织。第23届超大型数据库国际会议 (VLDB '97) 的会议记录。摩根考夫曼出版公司, 美国加利福尼亚州旧金山, 16–25。[20] 阿列克谢·科皮托夫。2019。可编写脚本的数据库和系统性能基准。https://github.com/akopytov/sysbench。[21] 托宾·J·雷曼和迈克尔·J·凯里。1986。主存数据库管理系统索引结构的研究。第12届超大型数据库国际会议 (VLDB '86) 论文集。摩根考夫曼出版公司, 美国加利福尼亚州旧金山, 294–303。[22] 维克托·莱斯、阿尔方斯·肯珀和托马斯·诺伊曼。2013。自适应基数树: 主内存数据库的 ARTful 索引。2013 年 IEEE 第29届国际数据工程会议 (ICDE)。IEEE, 38–49。[23] 贾斯汀·莱万多斯基、大卫·洛梅特和苏迪普塔·森古普塔。2013。LLAMA: 现代硬件的缓存/存储子系统。VLDB 捐赠基金会议记录 6, 10 (2013 年 8 月), 877–888。[24] 贾斯汀·莱万多斯基、大卫·洛梅特和苏迪普塔·森古普塔。2013。Bw-Tree: 用于新硬件平台的 B 树。2013 年 IEEE 第29届国际数据工程会议 (ICDE)。IEEE, 302–313。[25] MemSQL。2019。数据库基准测试工具。https://github.com/memsql/dbbench。[26] MySQL。2018。InnoDB 简介。https://dev.mysql.com/doc/refman/8.0/en/innodb-introduction.html。[27] 帕特里克·奥尼尔、爱德华·程、迪特·高利克和伊丽莎白·奥尼尔。1996。日志结构合并树 (LSM 树)。信息学报 33, 4 (1996), 351–385。[28] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram 和 Ittai Abraham。2017。PebblesDB: 使用碎片日志结构合并树构建键值存储。第26届操作系统原理研讨会 (SOSP '17) 论文集。ACM, 美国纽约州纽约市, 497–514。[29] 任凯, 郑庆, 乔伊·阿鲁拉吉, 加斯·吉布森。2017。SlimDB: 用于半排序数据的节省空间的键值存储引擎。VLDB 捐赠基金会议记录 10, 13 (2017 年 9 月), 2037–2048。[30] 孟德尔·罗森布鲁姆和约翰·K·奥斯特豪特。1992。日志结构文件系统的设计与实现。ACM 计算机系统汇刊 (TOCS) 10, 1 (1992 年 2 月), 26–52。[31] 拉塞尔·西尔斯和拉古·拉马克里希南。2012。bLSM: 通用日志结构化合并树。2012 年 ACM SIGMOD 国际数据管理会议 (SIGMOD '12) 会议记录。ACM, 美国纽约州纽约市, 217–228。[32] 滕德军, 郭雷, 李如宝, 陈峰, 马思远, 张艳峰, 张晓东。2017。LSbM-tree: 在数据管理中重新启用缓冲区缓存以进行混合读写。2017 年 IEEE 第37届分布式计算系统国际会议 (ICDCS)。IEEE, 68–79。[33] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurics, Tengiz Kharatishvili 和 Xiaofeng Bao。2017。Amazon Aurora: 高吞吐量云原生关系数据库的设计注意事项。2017 年 ACM 国际数据管理会议 (SIGMOD '17) 会议记录。ACM, 美国纽约, 1041–1052。[34] Hoang Tam Vo, Sheng Wang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi。2012。LogBase: 云中可扩展的日志结构数据库系统。VLDB 捐赠基金会议记录 5, 10 (2012 年 6 月), 1004–1015。

A 更多评价

在图 19 中，我们调整连接数量，使 X-Engine 在有和没有 FPGA 卸载的情况下以相同的 QPS 运行，然后测量并报告所有处理器的 CPU 利用率。无论是否有 FPGA 卸载，CPU 利用率平均分别约为 37 和 29 个硬件线程（回想一下，我们的 CPU 总共可满足 64 个线程，具有 32 个内核和超线程，因此最大利用率为 6400%）。通过将计算开销转移到 FPGA，FPGA 卸载还可将 CPU 利用率的方差降低约六倍。通过利用这些节省的 CPU 资源，X-Engine 能够将总体吞吐量提高 27%，如图 15 所示。

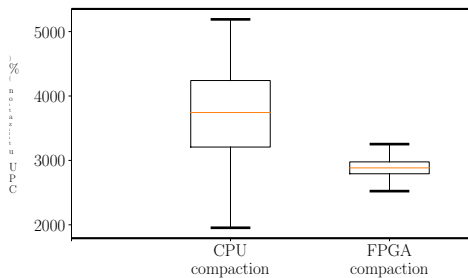


图 19：带有 X-Engine 的 MySQL 的 CPU 使用情况，使用 CPU 或 FPGA 进行压缩，同时提供相同水平的吞吐量。

图 20 绘制了运行写入密集型事务时每个事务读取的字节数和写入的字节数，比较了 RocksDB 和 X-Engine（两个基于 LSM 树的系统）。所有数据均根据 X-Engine 的结果进行归一化。由于 RocksDB 和 X-Engine 都使用索引来加速点查找，因此 X-Engine 的 extent 仅降低了约 9% 的读取放大。然而，这样的范围和索引有助于压缩中的数据重用，并有助于减少 63% 的写入放大。

B 生成电子商务工作负载

X-Bench 接受 SQL 工作负载的配置，其中我们指定了几种类型的事务。每笔交易都模仿某些电子商务操作，例如浏览给定类别中最热门的商品、下订单和付款。为了实现这一点，我们填写 SQL

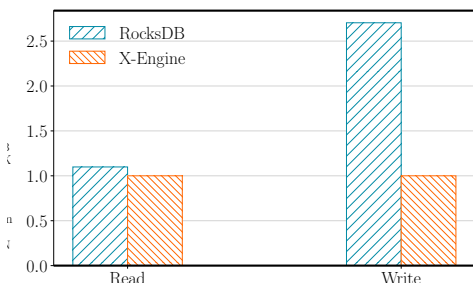


图 20：每个事务的标准化 KB 读取或写入。

以“更新”的形式将模板添加到每个交易中

TABLE_NAME SET PAY_STATUS = STATUS WHERE ORDER_ID = ID;”其中“TABLE_NAME”、“STATUS”和“ORDER_ID”是要填写的值的占位符。通过准备不同的 SQL 模板，我们能够在工作负载中生成点查找、范围查找、更新和插入。X-Bench 还允许配置每笔交易的权重，这样我们就可以改变每笔交易在工作负载中的百分比来模拟不同的工作负载。然后，X-Bench 调用 SysBench 根据指定的分布和其他条件用值填充这些占位符，最后使用这些合成查询来查询目标数据库实例。在阿里巴巴数据库管理员和用户的协助下，我们在评估中使用的 SQL 模板基于阿里巴巴电子商务平台的真实工作负载。

C 识别冷记录

如前所述，暖/冷层中的内部级别（如图 2 所示）通过数据温度（范围）进行区分。范围的温度是根据其在最近窗口中的访问频率来计算的。当进行 compaction 时，X-Engine 会选择阈值指定数量的最冷的 extent，比如 500 个 extent，并将这些 extent 推送到更深的层次进行 compaction。通过这样做，X-Engine 将温暖的区域保持在上层，将寒冷的区域保持在更深的区域。但这种方法不能很好地处理动态工作负载。例如，当某个范围的当前访问频率较低时，我们的算法会将该范围视为冷数据，但在不久的将来它可能会变成热数据。为此，我们研究了基于机器学习的算法来确定一定范围的适当温度。直觉是，除了范围之外，我们还使用行级别（记录）作为推断温度（暖/冷）的粒度。如果一条记录在最近的窗口中从未被访问过，则该记录被识别为冷记录。否则，它被认为是温暖的。因此，温度识别被转化为二元分类问题，并且可以使用分类模型来解决，例如使用随机森林或基于神经网络的方法。我们基于机器学习的温度识别算法的细节超出了本文的范围，将在另一项工作中详细阐述。

D 存储引擎和基准的配置

我们已经开源了用于使用 SysBench 和 dbbench online¹ 评估 InnoDB 和 RocksDB 的脚本。我们还分别在表 2、表 3 和表 4 中展示了评估中 MySQL (InnoDB)、MyRocks (RocksDB) 和 MySQL (X-Engine) 所使用的配置。表 2 和表 3 包含众所周知的配置选项。X-Engine 中的大多数选项与 RocksDB 和 InnoDB 类似。

¹https://github.com/x-engine-dev/test_scripts

Options	Values
innodb_buffer_pool_size	256G
innodb_doublewrite	1
innodb_flush_method	O_DIRECT
innodb_flush_log_at_trx_commit	1
innodb_log_file_size	2 GB
innodb_thread_concurrency	64
innodb_max_dirty_pages_pct_lwm	10
innodb_read_ahead_threshold	0
innodb_buffer_pool_instances	16
thread_cache_size	256
max_binlog_size	500 MB
read_buffer_size	128 KB
read_rnd_buffer_size	128 KB
table_open_cache_instances	16

表2: MySQL (InnoDB) 的配置

Options	Values
rocksdb_block_cache_size	170 GB
rocksdb_block_size	16384
rocksdb_max_total_wal_size	100 GB
rocksdb_max_background_jobs	15
rocksdb_max_subcompactions	1
target_file_size_base	256 MB
target_file_size_multiplier	1
level0_file_num_compaction_trigger	4
write_buffer_size	256 MB
max_write_buffer_number	4
max_bytes_for_level_multiplier	10
compression_per_level	No for all
num_levels	7 (default)
level_compaction_dynamic_level_bytes	True

表 3: MyRocks (RocksDB) 的配置

Options	Values
xengine_row_cache_size	45 GB
xengine_block_cache_size	170 GB
xengine_db_memtable_size	256 MB
xengine_db_total_memtable_size	100 GB
xengine_max_total_wal_size	100 GB
xengine_data_block_size	16384
xengine_max_compactions	8
xengine_max_flushes	3
xengine_max_memtable_number	2/sub-table
level0_extents_compaction_trigger	64
level1_extents_compaction_trigger	1000
xengine_compression	False
xengine_num_levels	3
xengine_thread_pool_size	128

表4: MySQL (X-Engine) 的配置

“level0_extents_compaction_trigger” 和 “level1_extents_compaction_trigger” 分别是 Level₀ 和 Level₁ 中的最大范围数。当达到这些阈值时会触发压缩。

最后, “xengine_max_compactions” 和 “xengine_max_flushes” 分别指的是X-Engine中运行的最大compaction线程和flush线程数。